

# GENIFY

## LECTURE 18

### 08-September-2025

#### TOPICS COVERED:

RIS

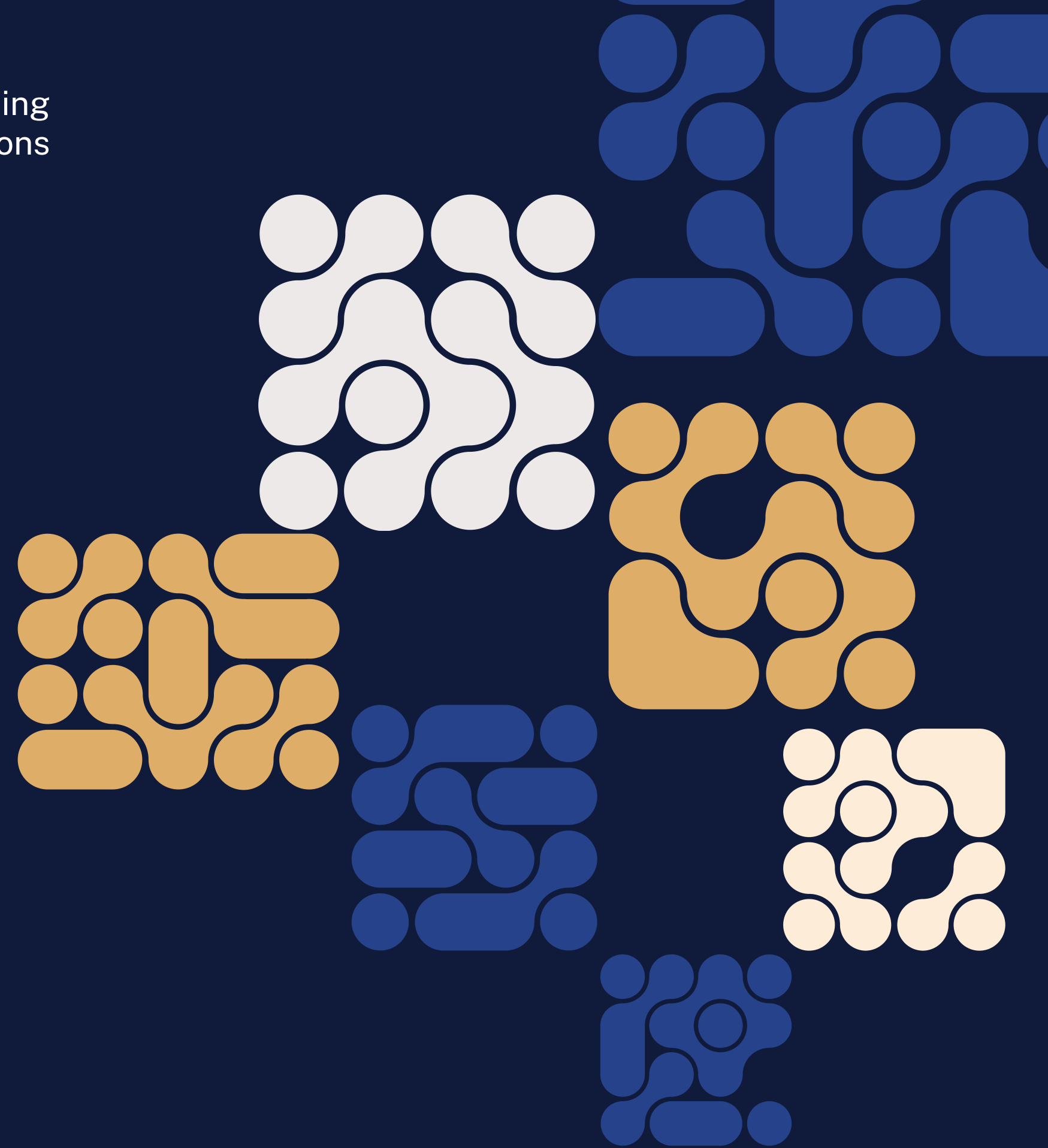
- LLM INFERENCING
- PROMPT ENGINEERING
- VECTOR DB

Presenter

**Shahzaib Kashif**

Co-Presenter

**Shayan Hassan Baig**

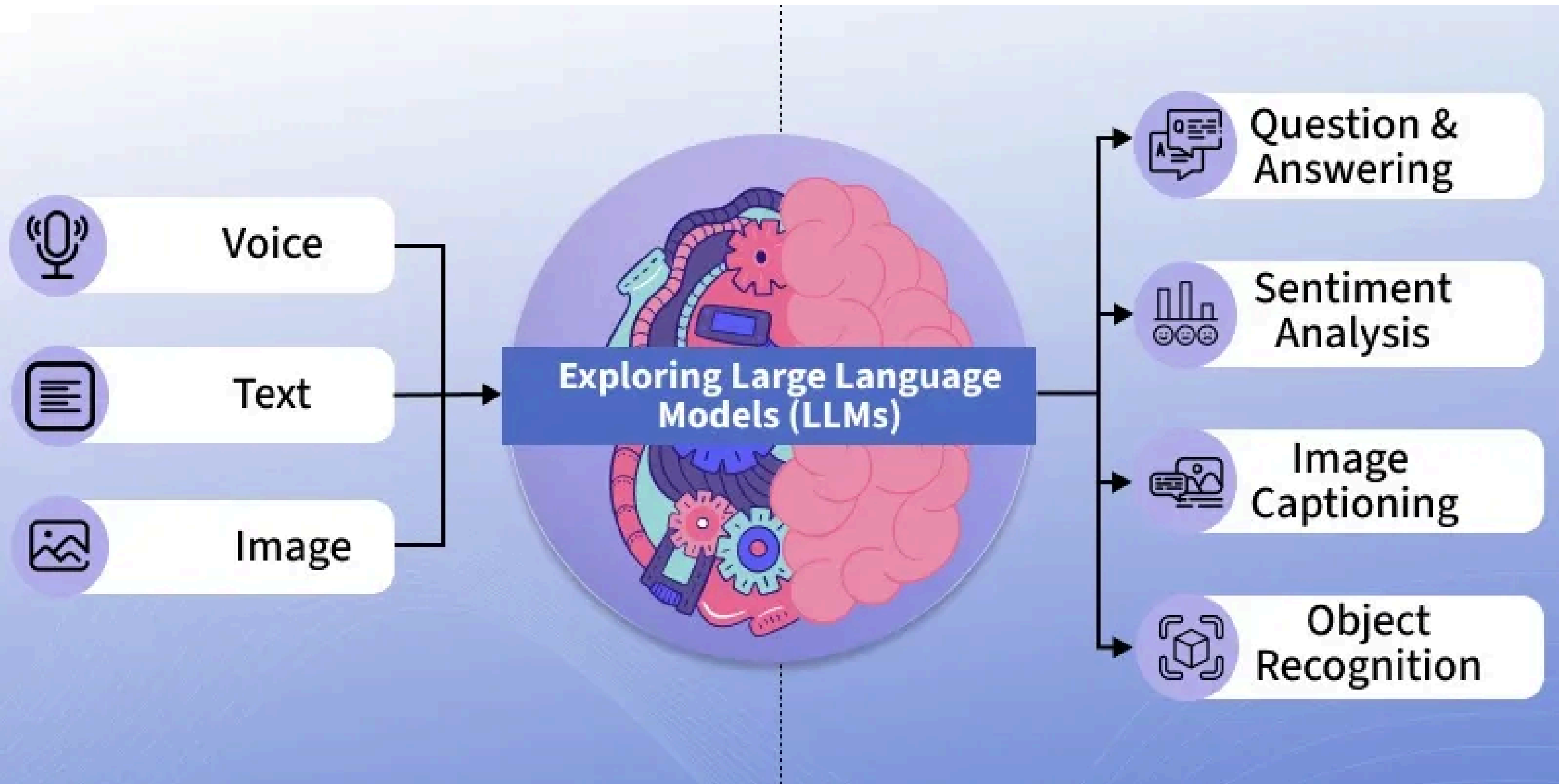




# LARGE LANGUAGE MODEL -RECAP

# Q: What's a LLM?

- Large Language Models (LLMs) are advanced AI systems based on deep neural networks, designed to process and generate text that resembles human language.
- They form the backbone of modern Natural Language Processing (NLP) applications, powering tools like ChatGPT, Google Gemini and Anthropic Claude.



# Evolution of GPT Models

2018



117 million  
parameters

2019



1.5 billion  
parameters

2020



175 billion  
parameters

2023



trillions of  
parameters

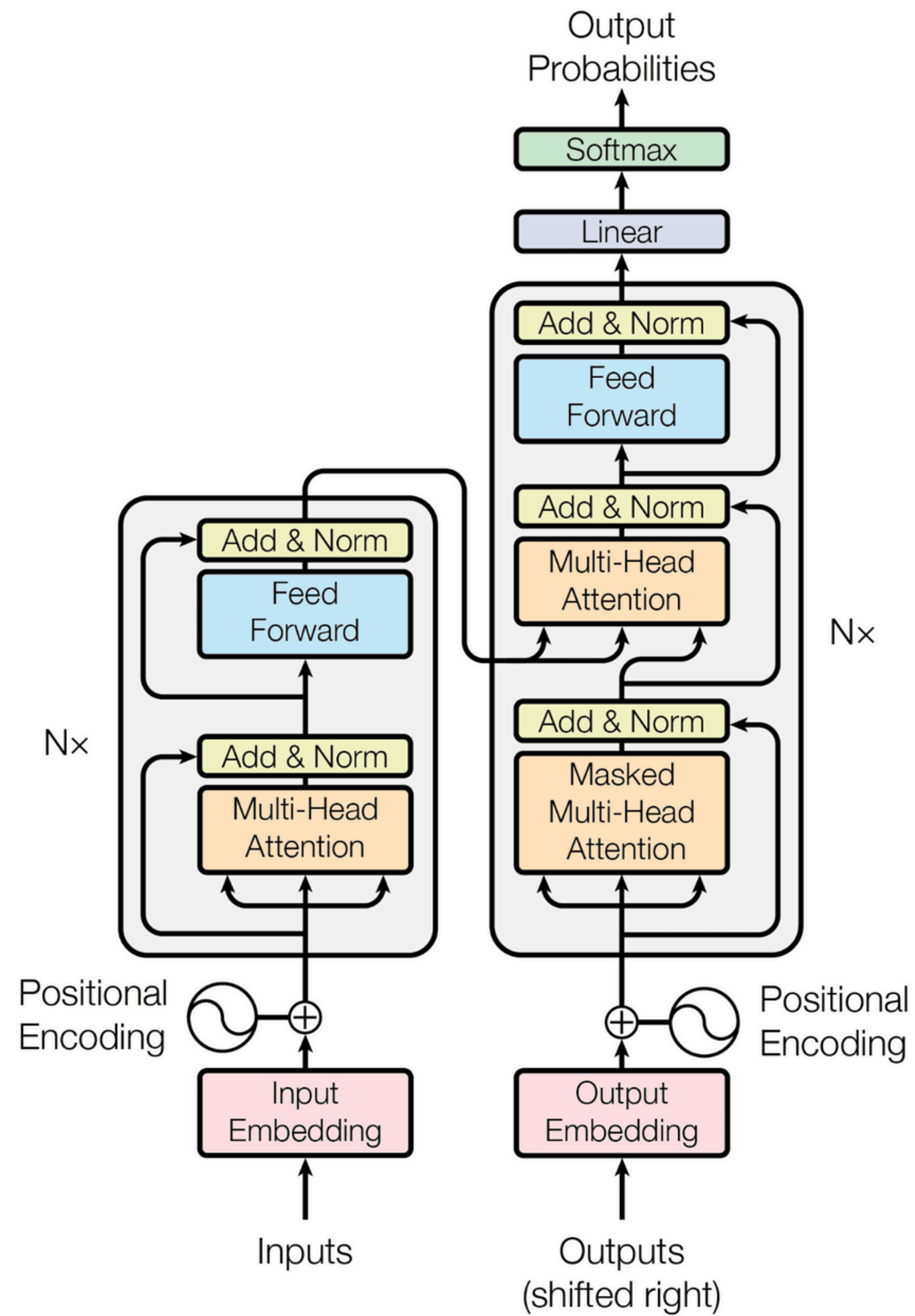
2023



optimized gpt-4

~985 million words





# Popular Large Language Models

- **GPT-4 & GPT-4o (OpenAI)**: Advanced multimodal reasoning.
- **Gemini 1.5 (Google DeepMind)**: Long-context reasoning (can process 1M+ tokens).
- **Claude 3 (Anthropic)**: Safety-focused, strong in reasoning and summarization.
- **LLaMA 3 (Meta)**: Open-weight model widely used in research & startups.
- **Mistral 7B / Mixtral (Mistral AI)**: Efficient open-source alternatives.
- **BERT / RoBERTa (Google/Facebook)**: Strong at embeddings and understanding tasks.
- **mBERT & XLM-R**: Early multilingual LLMs (contrary to claims that BLOOM was first).
- **BLOOM**: Collaborative open-source multilingual model (but not the first).

# Large Language Models Use Cases

- **Code Generation:** LLMs can generate accurate code based on user instructions for specific tasks.
- **Debugging and Documentation:** They assist in identifying code errors, suggesting fixes and even automating project documentation.
- **Question Answering:** Users can ask both casual and complex questions, receiving detailed, context-aware responses.
- **Language Translation and Correction:** LLMs can translate text between over 50 languages and correct grammatical errors.
- **Prompt-Based Versatility:** By crafting creative prompts, users can unlock endless possibilities, as LLMs excel in one-shot and zero-shot learning scenarios.



# Difference Between NLP and LLM

- NLP is Natural Language Processing, a field of artificial intelligence (AI).
- It consists of the development of the algorithms.
- NLP is a broader field than LLM, which consists of algorithms and techniques.
- NLP rules two approaches i.e. Machine learning and the analyze language data.

Applications of NLP are:

- Automotive routine task
  - Improve search
  - Search engine optimization
  - Analyzing and organizing large documents
  - Social Media Analytics.
- while on the other hand, LLM is a Large Language Model and is more specific to human- like text, providing content generation and personalized recommendations.

# Advantages of Large Language Models

Large Language Models (LLMs) come with several advantages that contribute to their widespread adoption and success in various applications:

- LLMs can perform zero-shot learning, meaning they can generalize to tasks for which they were not explicitly trained. This capability allows for adaptability to new applications and scenarios without additional training.
- LLMs efficiently handle vast amounts of data, making them suitable for tasks that require a deep understanding of extensive text corpora, such as language translation and document summarization.
- LLMs can be fine-tuned on specific datasets or domains, allowing for continuous learning and adaptation to specific use cases or industries.
- LLMs enable the automation of various language-related tasks, from code generation to content creation, freeing up human resources for more strategic and complex aspects of a project.

# Code Walkthrough Steps

```
[ ] prompt = "Explain gravity in one sentence"
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_length=50)
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

- Click on DropDown from the top right
- Select “Change runtime type”

Save

- Select “T4 GPU”
- Click Save



GPU Available: True

```
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
model = model.to("cuda") # Move to GPU
```

```
warnings.warn(
tokenizer_config.json: 2.54k/? [00:00<00:00, 70.6kB/s]
spiece.model: 100% 792k/792k [00:00<00:00, 274kB/s]
tokenizer.json: 2.42M/? [00:00<00:00, 8.90MB/s]
special_tokens_map.json: 2.20k/? [00:00<00:00, 81.8kB/s]
config.json: 1.40k/? [00:00<00:00, 47.6kB/s]
model.safetensors: 100% 308M/308M [00:02<00:00, 153MB/s]
generation_config.json: 100% 147/147 [00:00<00:00, 13.6kB/s]
```

```
[ ] prompt = "Explain gravity in one sentence"
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_length=50)
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

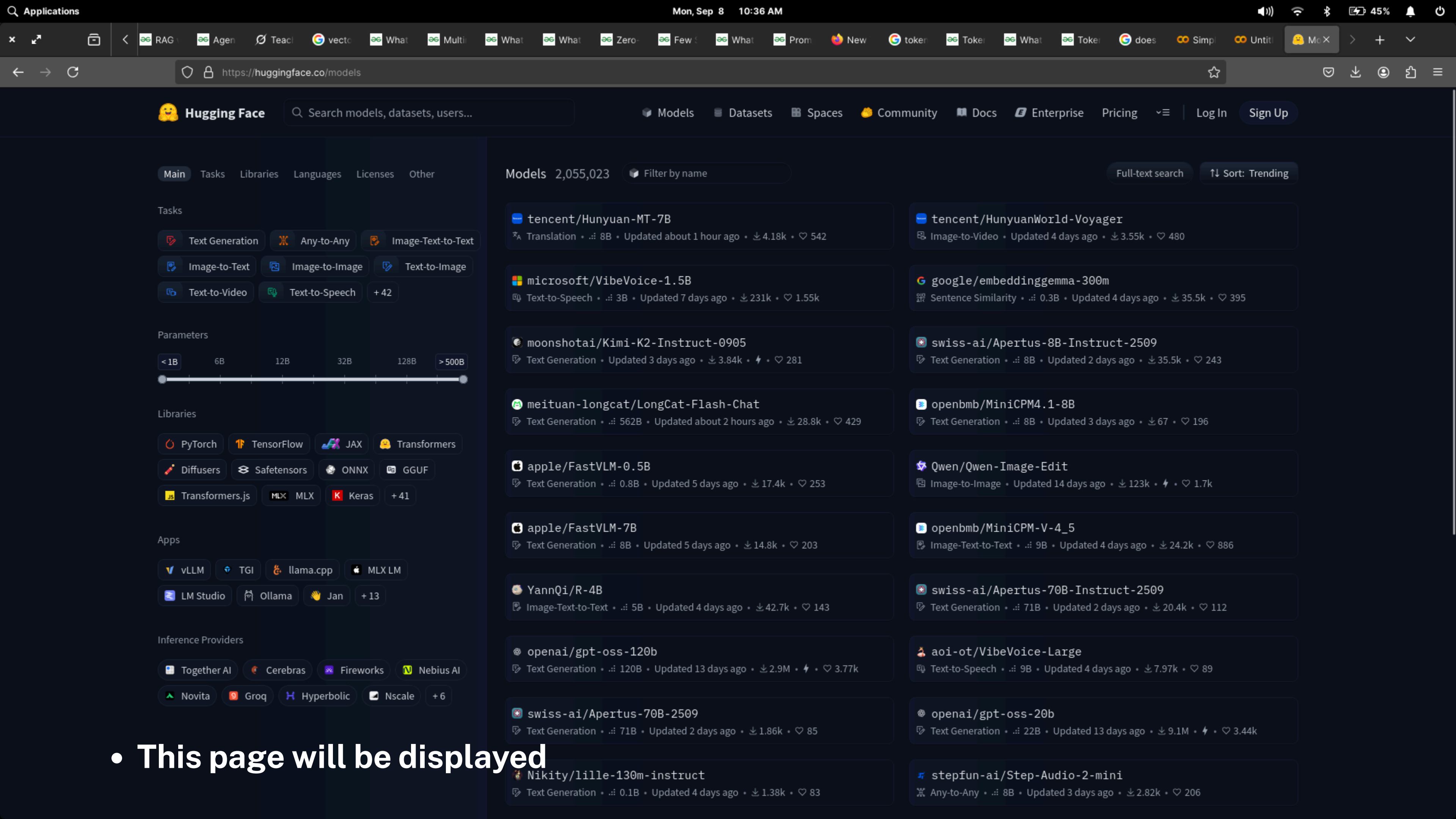
- Click “Connect” to connect to the GPU runtime

Code Walkthrough  
Follow this [Notebook](#)

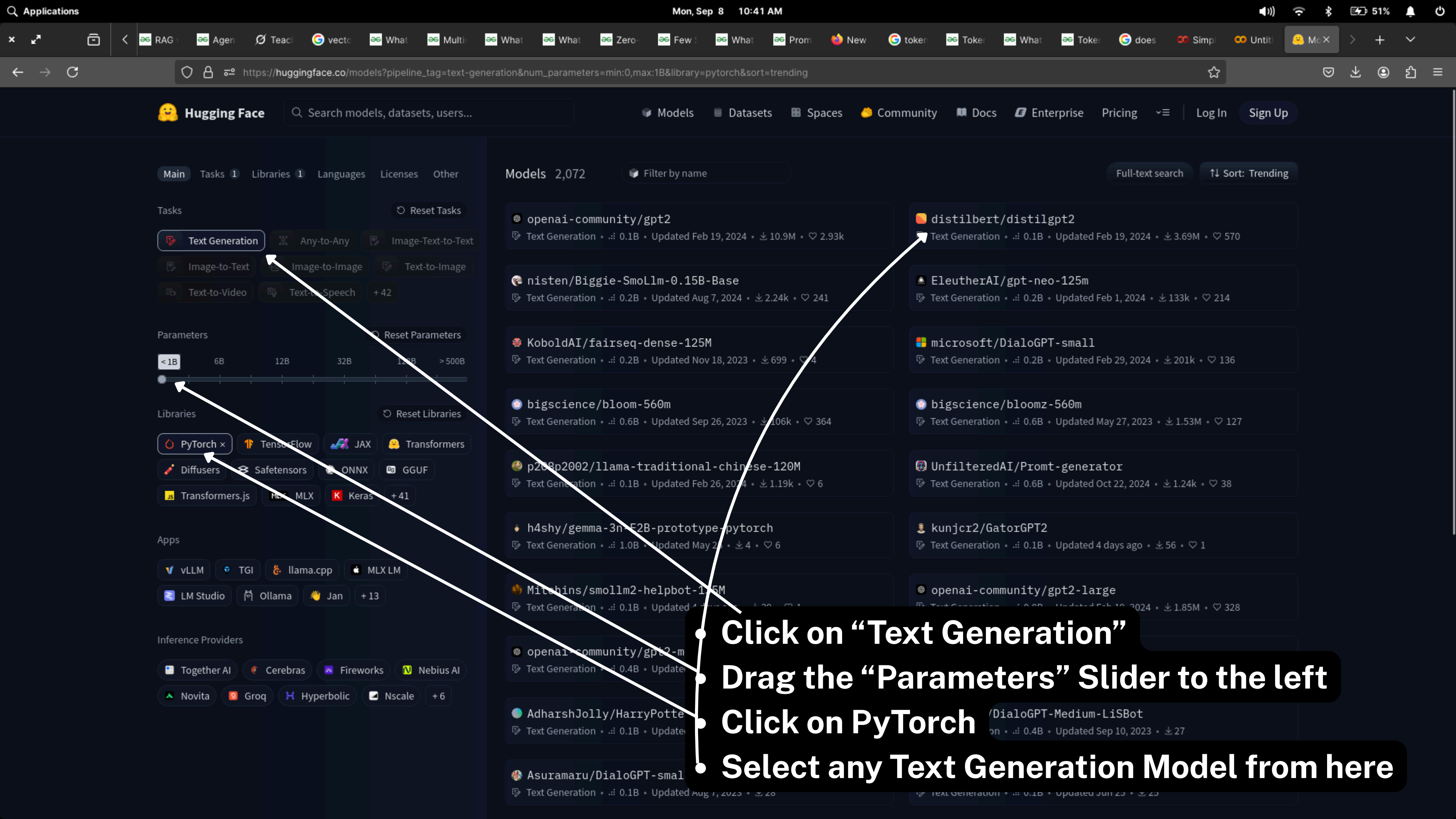








• This page will be displayed



Hugging Face

Search models, datasets, users...

Models

Datasets

Spaces

Community

Docs

Enterprise

Pricing

Log In

Sign Up

Main Tasks Libraries Languages Licenses Other

Tasks

Reset Tasks

Text Generation Any-to-Any Image-Text-to-Text

Image-to-Text Image-to-Image Text-to-Image

Text-to-Video Text-to-Speech + 42

Parameters

Reset Parameters

< 1B 6B 12B 32B 128B > 500B

Libraries

Reset Libraries

PyTorch TensorFlow JAX Transformers

Diffusers Safetensors ONNX GGUF

Transformers.js MLX Keras + 41

Apps

vLLM TGI llama.cpp MLX LM

LM Studio Ollama Jan + 13

Inference Providers

Together AI Cerebras Fireworks Nebius AI

Novita Groq Hyperbolic Nscale + 6

Models 2,072

Filter by name

Full-text search

Sort: Trending

openai-community/gpt2

Text Generation · 0.1B · Updated Feb 19, 2024 · 10.9M · 2.93k

nisten/Biggie-SmolLM-0.15B-Base

Text Generation · 0.2B · Updated Aug 7, 2024 · 2.24k · 241

KoboldAI/fairseq-dense-125M

Text Generation · 0.2B · Updated Nov 18, 2023 · 699 · 34

bigscience/bloom-560m

Text Generation · 0.6B · Updated Sep 26, 2023 · 106k · 364

p208p2002/llama-traditional-chinese-120M

Text Generation · 0.1B · Updated Feb 26, 2024 · 1.19k · 6

h4shy/gemma-3n-52B-prototype-pytorch

Text Generation · 1.0B · Updated May 25, 2024 · 4 · 6

Mitchins/smollm2-helpbot-1.5M

Text Generation · 0.1B · Updated Apr 1, 2024 · 1.38k · 31

openai-community/gpt2-medium

Text Generation · 0.4B · Updated Feb 19, 2024 · 1.85M · 328

AdharshJolly/HarryPotterGPT

Text Generation · 0.1B · Updated Sep 10, 2023 · 27

Asuramaru/DialogGPT-small

Text Generation · 0.1B · Updated Aug 1, 2023 · 20

distilbert/distilgpt2

Text Generation · 0.1B · Updated Feb 19, 2024 · 3.69M · 570

EleutherAI/gpt-neo-125m

Text Generation · 0.2B · Updated Feb 1, 2024 · 133k · 214

microsoft/DialogGPT-small

Text Generation · 0.2B · Updated Feb 29, 2024 · 201k · 136

bigscience/bloomz-560m

Text Generation · 0.6B · Updated May 27, 2023 · 1.53M · 127

UnfilteredAI/Prompt-generator

Text Generation · 0.6B · Updated Oct 22, 2024 · 1.24k · 38

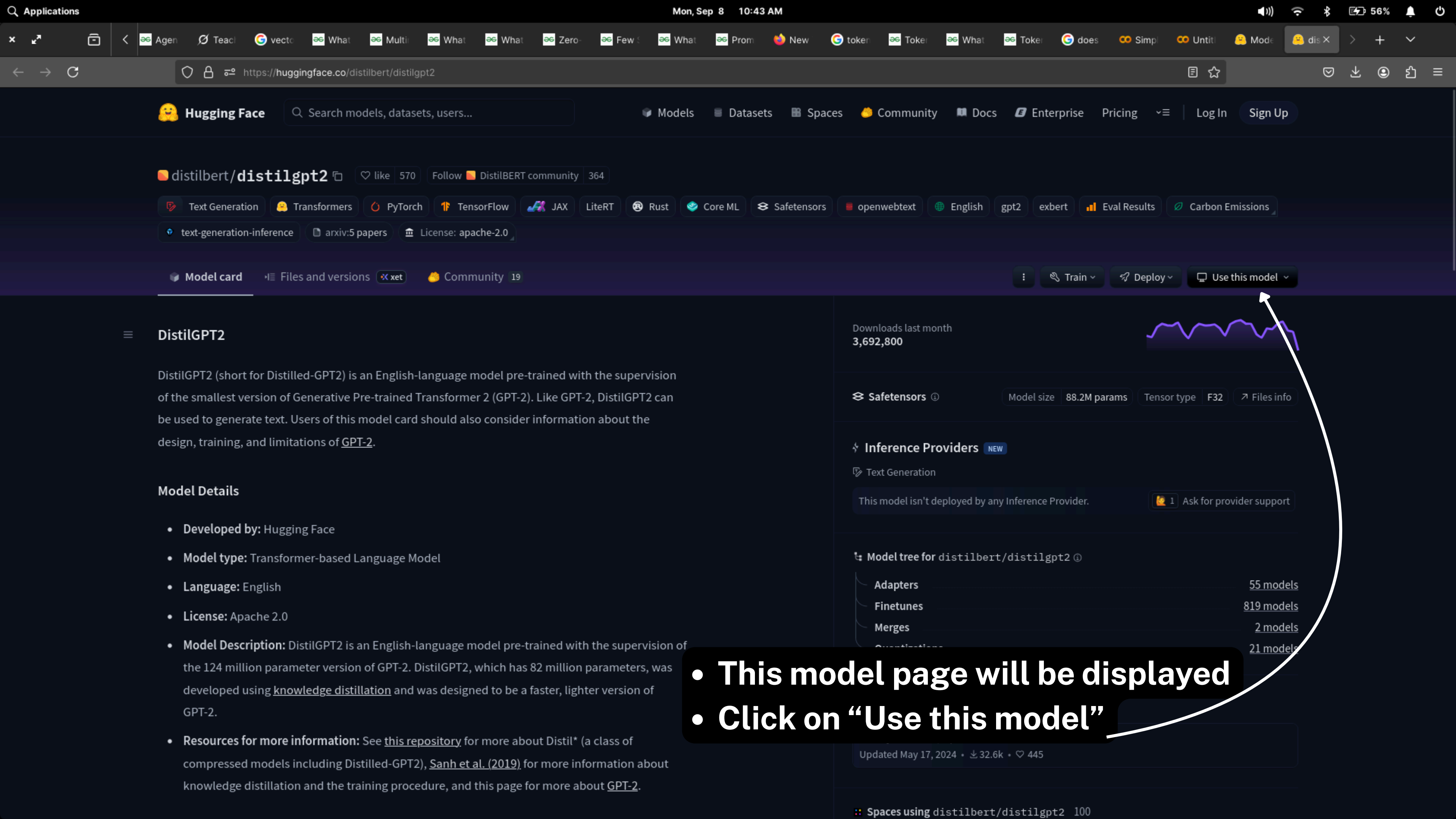
kunjcr2/GatorGPT2

Text Generation · 0.1B · Updated 4 days ago · 56 · 1

openai-community/gpt2-large

Text Generation · 0.7B · Updated Feb 19, 2024 · 1.85M · 328

- Click on “Text Generation”
- Drag the “Parameters” Slider to the left
- Click on PyTorch
- Select any Text Generation Model from here



distilbert/distilgpt2 like 570 Follow DistilBERT community 364

Text Generation Transformers PyTorch TensorFlow JAX LiteRT Rust Core ML Safetensors openwebtext English gpt2 exbert Eval Results Carbon Emissions text-generation-inference arxiv:5 papers License: apache-2.0

Model card Files and versions xet Community 19

Train Deploy Use this model

## DistilGPT2

DistilGPT2 (short for Distilled-GPT2) is an English-language model pre-trained with the supervision of the smallest version of Generative Pre-trained Transformer 2 (GPT-2). Like GPT-2, DistilGPT2 can be used to generate text. Users of this model card should also consider information about the design, training, and limitations of [GPT-2](#).

### Model Details

- Developed by: Hugging Face
- Model type: Transformer-based Language Model
- Language: English
- License: Apache 2.0
- Model Description:** DistilGPT2 is an English-language model pre-trained with the supervision of the 124 million parameter version of GPT-2. DistilGPT2, which has 82 million parameters, was developed using [knowledge distillation](#) and was designed to be a faster, lighter version of GPT-2.
- Resources for more information:** See [this repository](#) for more about Distil\* (a class of compressed models including Distilled-GPT2), [Sanh et al. \(2019\)](#) for more information about knowledge distillation and the training procedure, and this page for more about [GPT-2](#).

Downloads last month  
3,692,800



Safetensors Model size 88.2M params Tensor type F32 Files info

### Inference Providers

Text Generation

This model isn't deployed by any Inference Provider.

1 Ask for provider support

### Model tree for distilbert/distilgpt2

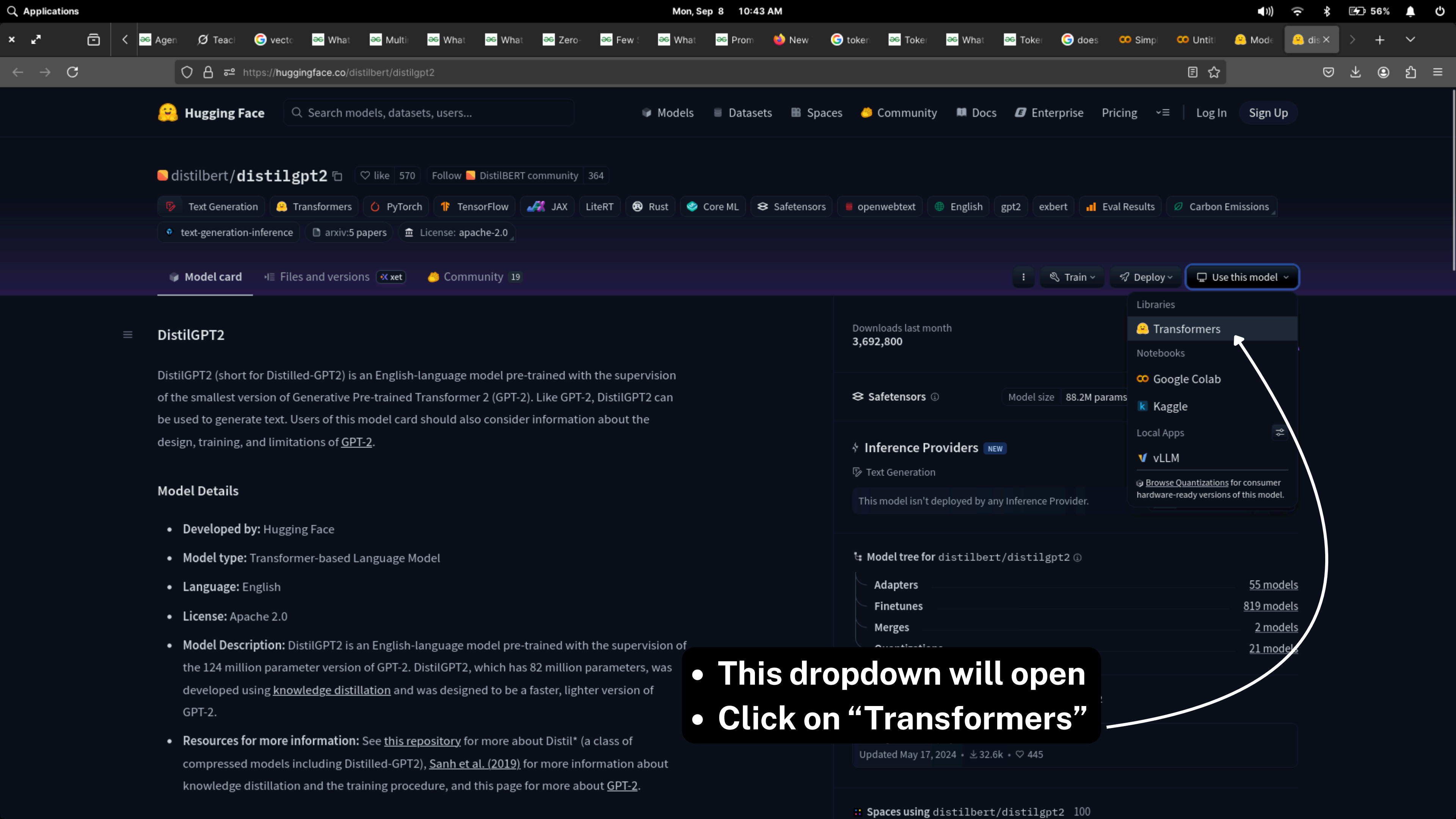
Adapters 55 models  
Finetunes 819 models  
Merges 2 models  
Quantizations 21 models

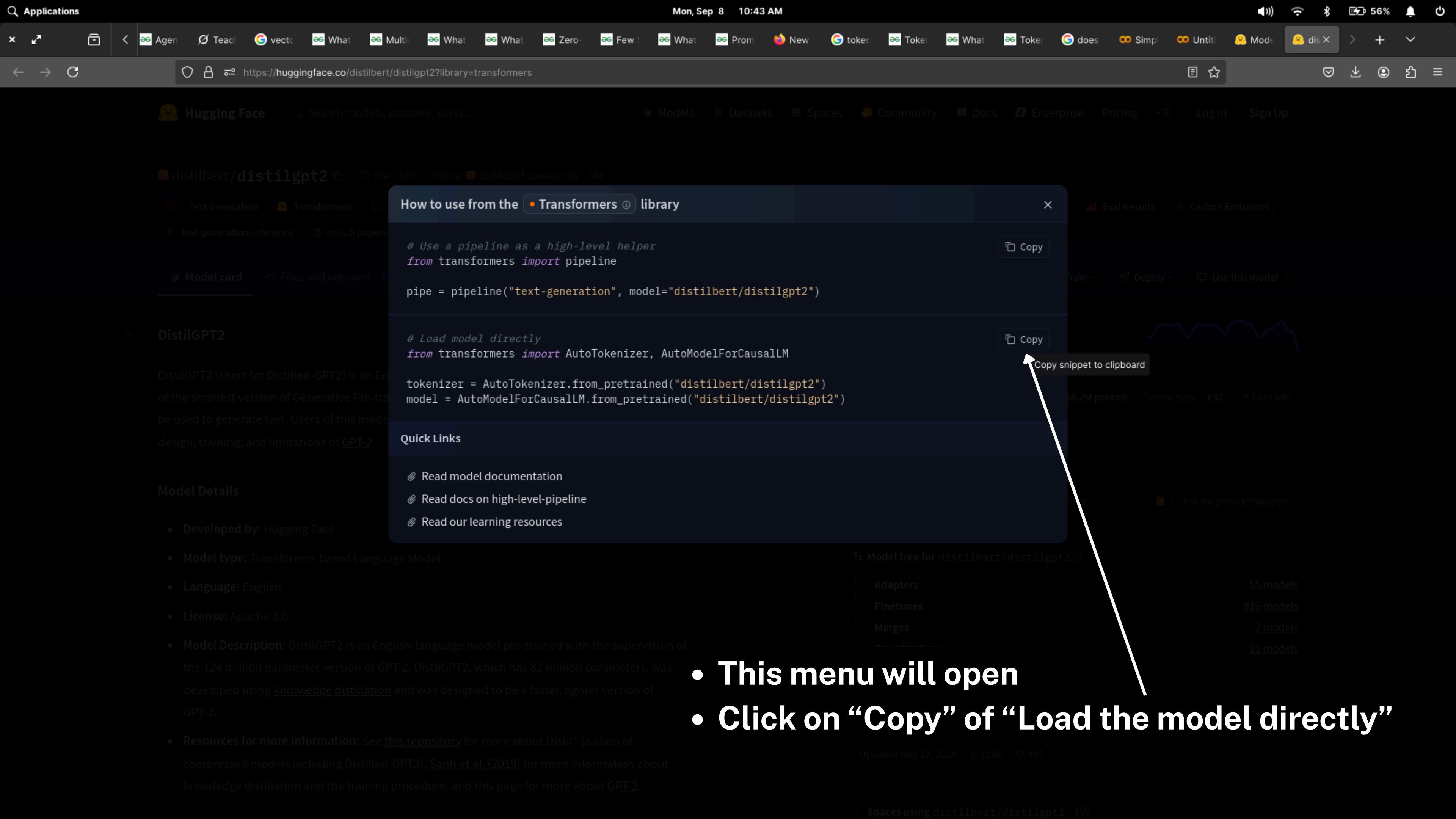
Updated May 17, 2024 • 32.6k • 445

Spaces using distilbert/distilgpt2 100

- This model page will be displayed
- Click on “Use this model”







### How to use from the **Transformers** library

*# Use a pipeline as a high-level helper*  
`from transformers import pipeline`

Copy

`pipe = pipeline("text-generation", model="distilbert/distilgpt2")`

*# Load model directly*  
`from transformers import AutoTokenizer, AutoModelForCausalLM`

Copy

`tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")`  
`model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")`

Copy snippet to clipboard

#### Quick Links

- [Read model documentation](#)
- [Read docs on high-level-pipeline](#)
- [Read our learning resources](#)

- This menu will open
- Click on “Copy” of “Load the model directly”

Applications

Mon, Sep 8 10:43 AM

56%

Agen

Teacl

vecto

What

Multi

What

What

Zero

Few

What

Prom

New

token

Token

What

Token

does

Sin X

Un titl

Mod

distill

+

✓

←

→

↺

https://colab.research.google.com/drive/1kF7tRM-Biq39dtt\_7jV9Qh\_oti3ps1Um?authuser=2#scrollTo=uSZi7WCO1tpw

📄

★

📧

⬇

👤

📁

☰

CO

SimpleLLMInference.ipynb

☆

☁

File

Edit

View

Insert

Runtime

Tools

Help

🔍

Commands

+ Code

+ Text

▶ Run all

▼

0s

▶

# Load model directly

from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from\_pretrained("distilbert/distilgpt2")

model = AutoModelForCausalLM.from\_pretrained("distilbert/distilgpt2")

model.to("cuda")

↔

GPT2LMHeadModel(

(transformer): GPT2Model(

(wte): Embedding(50257, 768)

(wpe): Embedding(1024, 768)

(drop): Dropout(p=0.1, inplace=False)

(h): ModuleList(

(0-5): 6 x GPT2Block(

(ln\_1): LayerNorm((768,), eps=1e-05, elementwise\_affine=True)

(attn): GPT2Attention(

(c\_attn): Conv1D(nf=2304, nx=768)

(c\_proj): Conv1D(nf=768, nx=768)

(attn\_dropout): Dropout(p=0.1, inplace=False)

(resid\_dropout): Dropout(p=0.1, inplace=False)

)

(ln\_2): LayerNorm((768,), eps=1e-05, elementwise\_affine=True)

(mlp): GPT2MLP(

(c\_fc): Conv1D(nf=3072, nx=768)

(c\_proj): Conv1D(nf=768, nx=3072)

(act): NewGELUActivation()

(dropout): Dropout(p=0.1, inplace=False)

)

)

(ln\_f): LayerNorm((768,), eps=1e-05, elementwise\_affine=True)

)

(lm\_head): Linear(in\_features=768, out\_features=50257, bias=False)

)

0s

[14]

prompt = "Tell me about RISC-V ISA"

inputs = tokenizer(prompt, return\_tensors="pt").to("cuda")

outputs = model.generate(\*\*inputs, max\_length=50)

response = tokenizer.decode(outputs[0], skip\_special\_tokens=True)

print("LLM Response:", response)

↔

Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

LLM Response: Tell me about RISC-V ISA, and I'm not sure if I'm going to be able to do it.'

📄

Variables

Terminal

10:42 AM

T4 (Python 3)

• Paste the copied line here and run the cell

• Paste the code from previous cell to run a prompt

• Write any prompt here you like

• Experiment with different Text generation models and evaluate how accurate they generate text or answer



# PROMPT ENGINEERING

# What is Prompt Engineering

## Meaning, Working, Techniques

- Prompt engineering is rapidly emerging as a critical skill in the age of Artificial Intelligence (AI).
- As AI continues to revolutionize various fields, prompt engineering empowers us to extract the most value from these powerful models.
- This comprehensive guide dives deep into the world of prompt engineering, exploring its core principles, applications, and best practices.



# Q: What is Prompt Engineering?

- Prompt engineering is the process of creating effective prompts that enable AI Models to generate responses based on given inputs.
- Prompt engineering essentially means writing prompts intelligently for text-based Artificial Intelligence tasks, more specifically, Natural Language Processing (NLP) tasks.
- In the case of such text-based tasks, these prompts help the user and the model generate a particular output as per the requirement.
- These requirements are efficiently added in the form of prompts and hence the name Prompt Engineering.

# Q: What are Prompts?

- Prompts are short pieces of text that are used to provide context and guidance to machine learning models.
- When talking about the specific text AI tasks, also called NLP tasks, these prompts are useful in generating relevant outputs which are as close to the expected output itself.
- Precisely, these prompts help in generating accurate responses by:
  - Adding on some additional guidance for the model.
  - Not generalizing a prompt too much.
  - Make sure the information added is not too much as that can confuse the model.
  - Making the user intent and purpose clear for the model to generate content in the relevant context only.

# Q: Prompt Engineering: Why is it Important?

- More specific formats of input as prompts help in better interpretability of the requirements for a task.
- Specific prompts with a detailed explanation of the requirements mean output matches more with the desired one.
- Better results for NLP tasks, through prompts also mean a better-trained model for future tasks.

# Q: How Prompt Engineering Works?

- Imagine you're instructing a very talented but inexperienced assistant.
- You want them to complete a task effectively, so you need to provide clear instructions.
- Prompt engineering is similar, it's about crafting the right instructions, called prompts, to get the desired results from a large language model (LLM).
- Working of Prompt Engineering Involves:
  - **Crafting the Prompt:** You design a prompt that specifies what you want the LLM to do. This can be a question, a statement, or even an example. The wording, phrasing, and context you include all play a role in guiding the LLM's response.
  - **Understanding the LLM:** Different prompts work better with different LLMs. Some techniques involve giving the LLM minimal instructions (zero shot prompting), while others provide more context or examples (few shot prompting).
  - **Refining the Prompt:** It's often a trial-and-error process. You might need to tweak the prompt based on the LLM's output to get the kind of response you're looking for.

# Q: What are Prompt Engineering Techniques?

The purpose of the prompt engineering is not limited to the drafting of prompts. It is a playground that has all the tools to adjust your way of working with the big language models (LLMs) with specific purposes in mind.



# Q: What are Prompt Engineering Techniques?

## Foundational Techniques

- **Information Retrieval:** This entails the creation of prompts so that the LLM can get its knowledge base and give out what is relevant.
- **Context Amplification:** Give supplementary context to the prompt in order to direct the understanding and attention of the LLM to its output.
- **Summarization:** Induce the LLM to generalize or write summaries about complex themes.
- **Reframing:** Rephrase your reminder to the LLM to consider a specific style or format for the output.
- **Iterative Prompting:** Break down the complex tasks into smaller parts and then instruct the LLM sequentially in how to achieve the end result.

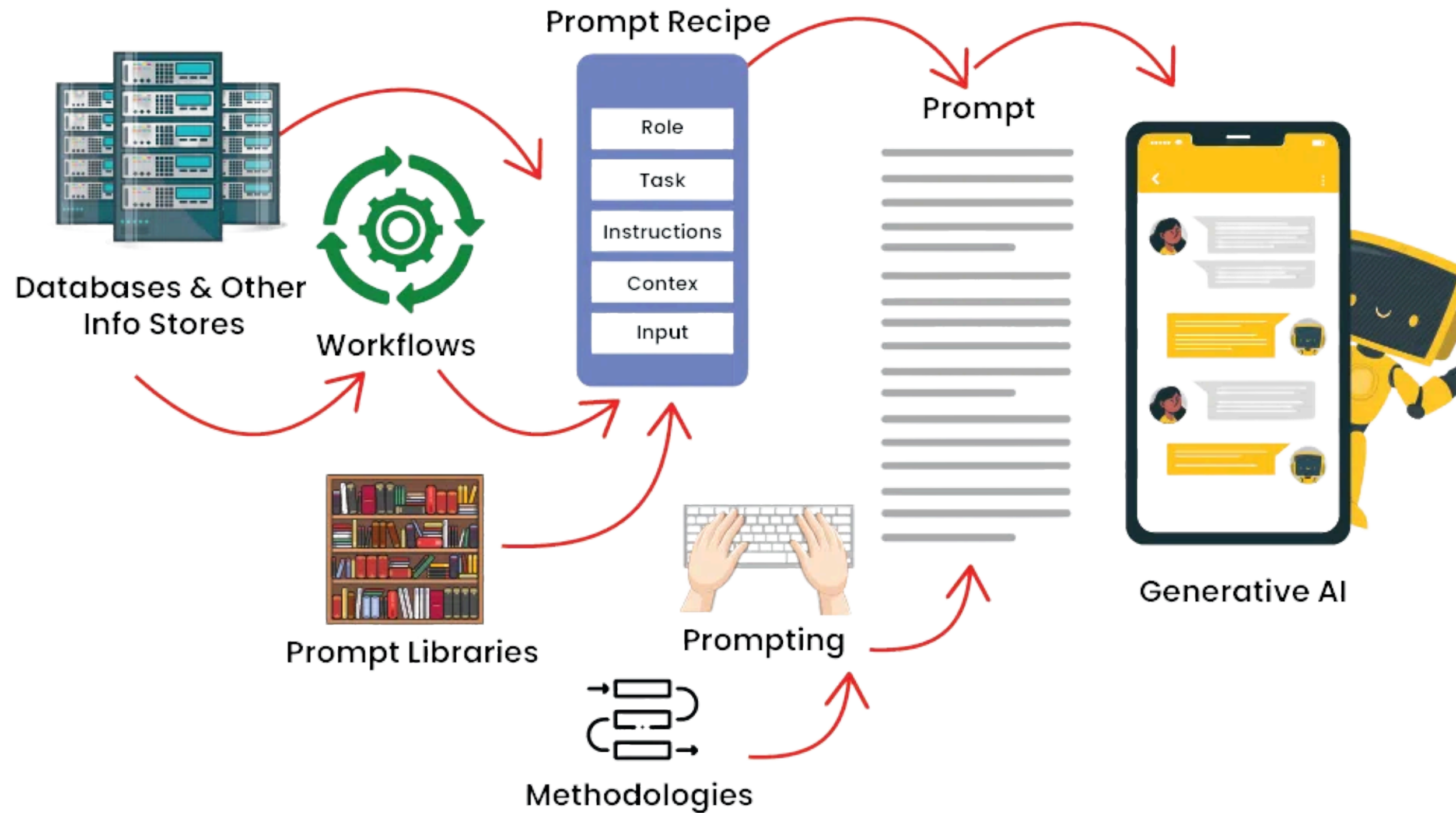


# Q: What are Prompt Engineering Techniques?

## Advanced Techniques

- **Least to Most Prompting:** First, begin with prompts of general nature and then add facts to drive the LLM to make a highly specialized solution for intricate problems.
- **Chain-of-Thought (CoT) Prompting:** Require the LLM to show the steps of its reasoning as well as the answer, leading to enlightenments for our understanding of its thinking.
- **Self-Ask Prompting:** This thus entails chaining of thought (CoT) prompting, which involved the LLM being prompted to ask itself clarifying questions to get to a solution.
- **Meta-Prompting:** This experimental method investigates designing a single, common prompt that can be used for diverse tasks by way of additional instructions.

# What is an AI Prompt Engineering?



# Techniques for AI Prompt Engineering

| Technique            | Description                                                                  | Example Use Case                          |
|----------------------|------------------------------------------------------------------------------|-------------------------------------------|
| Contextual Prompts   | Provide background or context to guide the AI's response                     | “What is the weather in Karachi today?”   |
| Specificity          | Use precise language to reduce ambiguity                                     | “Explain the impact of AI on healthcare.” |
| Iterative Refinement | Continuously test and adjust prompts to improve results                      | Refine a prompt until output is accurate  |
| Prompt Templates     | Use standardized formats for consistency across similar tasks                | Templates for FAQ bots                    |
| Experimentation      | Try different prompt types (open-ended, closed-ended) to see what works best | Compare “Describe X” vs. “List X facts”   |